

Project 4 – Classification Image TTT4275 Estimation, Detection and Classification

Oskar Nysæther

Isak Kvammen

April 23, 2026

Abstract

This classification project discusses linear and template classifiers, used for classifying the Iris flower dataset and hand written numbers from the MNIST database, respectively. For the Iris task, a linear classifier was implemented and trained using minimum square error and gradient descent. The results show that the classifier shows good performance, with most errors occurring between overlapping classes, while one of the classes is completely separable.

For the MNIST task, three template-based classifiers were evaluated: nearest neighbor (NN), nearest neighbor with k-means clustering and k-nearest neighbors (KNN). The NN classifier achieves an accuracy of 96.91% with a high computational cost. Clustering lower the computational cost significantly at the expense of a moderate drop in accuracy, to 95.23%. The KNN classifier with $k = 7$ achieved the highest accuracy of 96.94%, but also required the longest runtime. The results demonstrated clear trade-offs between accuracy and computational efficiency. Overall, the study highlights how model complexity and data representation affect classification performance.

Contents

1	Introduction	2
2	Theory	2
2.1	Linear classifier	2
2.2	Sigmoid transformation	2
2.3	MSE	3
2.4	The Weight matrix and the step factor α	3
2.5	Confusion matrix	3
2.6	Error rate	4
2.7	Linear separability	4
2.8	Nearest Neighbor estimator	4
2.9	K-NN classifier	4
2.10	Clustering	4
3	The Iris Task	5
3.1	Task description	5
3.2	Part 1: Design, training, and generalization	5
3.3	Part 2. Features and linear separability	6
3.4	Conclusion	9
4	Handwritten numbers	9
4.1	Task description	9
4.2	Implementation and results	9
4.3	Discussion	12
4.4	Conclusion	12

1 Introduction

Classification is an important problem in statistical learning and pattern recognition. The objective is to assign observations to the correct class based on measured features.

This report presents two classification problems with different solution methods. The first problem involves classification of Iris flowers based on measured features, where a classifier is used to separate the samples into groups. This problem is known to be close to linearly separable and is therefore well suited for a linear classifier.

The second problem is handwritten digit recognition using the MNIST dataset, where nearest neighbor methods and clustering techniques are applied.

The report is organized as follows. First, the relevant theory is presented. Then, the Iris task is described, implemented, analyzed, and concluded. Finally, the MNIST task is described, implemented, analyzed, and concluded.

2 Theory

This chapter presents the relevant theory needed to understand the tasks.

2.1 Linear classifier

A linear classifier is a classifier that uses a linear decision boundary to separate the different groups.

For a classification problem with C classes and d features, the output of a linear classifier can be written as a discriminant vector

$$g(x) = Wx + w_0,$$

where $W \in \mathbb{R}^{C \times d}$ is the weight matrix, $w_0 \in \mathbb{R}^C$ is the bias vector, and $x \in \mathbb{R}^d$ is the input vector.

This expression can be rewritten by augmenting the input vector with a bias term. By defining

$$\tilde{x} = [x \ 1], \quad \tilde{W} = [W; w_0],$$

the classifier can then be written in a more compact form as

$$g(x) = \tilde{W}\tilde{x}.$$

When a sigmoid transformation is applied to the linear classifier, the output becomes

$$g(x) = \sigma(\tilde{W}\tilde{x}).$$

2.2 Sigmoid transformation

The output from a linear classifier can take any real value. In some cases, the sigmoid function is used to transform this output into a value between 0 and 1.

This makes the output easier to interpret as the probability of belonging to a given class.

The sigmoid function is given by:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The function is especially useful in classification problems, since values close to 0 indicate low probability and values close to 1 indicate high probability.

2.3 MSE

Mean square error (MSE) is a method of training classifiers. The objective is to minimize the error between the classifier output and the target value, given by:

$$MSE = \frac{1}{2} \sum_{k=1}^N (g_k - t_k)^T (g_k - t_k)$$

Here, g_k represents the classifier output and t_k represents the target value. [1], p. 16

Using the chain rule, the gradient of the MSE with respect to the weight matrix can be expressed as

$$\nabla_W MSE = \sum_{k=1}^N ((g_k - t_k) \circ g_k \circ (1 - g_k)) \tilde{x}_k^T,$$

where $\circ$ denotes element-wise multiplication. This expression corresponds to the gradient used in the implementation.

2.4 The Weight matrix and the step factor α

The weight matrix W contains the classifier parameters and is updated iteratively during training in order to minimize the classification error.

This is given by:

$$W(m) = W(m-1) - \alpha \nabla_W MSE$$

The gradient of the MSE with respect to the weight matrix is used because it indicates how the error changes with respect to the model parameters.

By subtracting the gradient, the weight matrix is updated in the direction that reduces the error the fastest, so the classifier converges towards an optimal solution.

The step factor α is the learning rate and determines the step length in the gradient descent. It controls how much the error from the MSE contributes to the update of the weight matrix W , and must be chosen appropriately.

2.5 Confusion matrix

The confusion matrix is a tool used to evaluate the performance of a classifier. It gives a summary of how many data points have been classified into different groups, and whether they have been classified correctly.

The diagonal elements represent correctly classified samples, while the off-diagonal elements represent misclassified samples.

True class/Predicted class	H_1	H_2	H_3
H_1	$P_{1,1}$	$P_{1,2}$	$P_{1,3}$
H_2	$P_{2,1}$	$P_{2,2}$	$P_{2,3}$
H_3	$P_{3,1}$	$P_{3,2}$	$P_{3,3}$

Table 1: Confusion matrix for a classifier

A perfect classifier gives a confusion matrix with all values centered on the diagonal.

2.6 Error rate

The error rate is often given as a percentage that indicates how many data points have been classified into an incorrect group. It is given by:

$$\text{Error rate} = \frac{\text{number of misclassified samples}}{\text{total number of samples}} \cdot 100\%$$

2.7 Linear separability

When we have linear separability it is possible to separate groups without errors with simple lines. [1], p. 6

2.8 Nearest Neighbor estimator

A nearest neighbor estimator calculates the distance to each of the data points in the known template set, and assumes the instance belongs to the same class as the nearest neighbor. There are multiple types of nearest neighbor estimators, depending on what measurement is used for distance.[1, p. 10] The simplest of these uses Euclidean distance:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2$$

This is often implemented through libraries that compute the pairwise distance matrix between the two sets efficiently. An example of this is the `cdist` function from `scipy`. It computes the pairwise Euclidean distances and returns them as a distance matrix. [2]

2.9 K-NN classifier

The K-NN classifier works on the same principle as the NN classifier, but looks at the K nearest neighbors. The decision rule of the K-NN classifier takes into account the most frequent class and selects this as the predicted class.[1, p. 10]

$$c(x) = \arg \max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(x)} \mathbf{1}(y_i = c)$$

2.10 Clustering

The simplest way to build a template estimator is to use all the available training-data as a template set. However when N data points and M features grow, the full nearest neighbor computation becomes expensive very fast. One way to handle this is to use clusters instead of the full dataset. A cluster is a central point used to represent the neighboring data points. The cluster should be positioned such that all points in the same group are similar, and different groups are relatively dissimilar. [1, p. 18]

The K-Means algorithm is an example of a clustering algorithm. It aims to minimize the following objective function:

$$\sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

The algorithm works by iterating, and has the following steps: [1]

1. **Input:** Data points $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ and number of clusters k .
2. Initialize centroids $\{\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k\}$.

3. **Assignment step:** For each point, assign it to the nearest centroid:

$$c_i = \arg \min_{j \in \{1, \dots, k\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

4. **Update step:** Recompute centroids as the mean of assigned points:

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i$$

5. Repeat steps 3–4 until convergence (no change in assignments or centroids).

6. **Output:** Clusters $C_1, \dots, C_k$.

This yields a template set consisting of centroids for the clusters that represents the original set.

3 The Iris Task

The Iris task is based on a Fisher Iris data set, which consists of 50 examples of each of three classes of Iris flowers: Setosa, Versicolor, and Virginica. Each sample is described by four measured features: sepal length, sepal width, petal length, and petal width.

The objective of the task is to classify the flowers into the correct groups using a linear classifier and to analyze how different features affect the classifier’s performance.

3.1 Task description

In Part 1, a linear classifier is trained using two different train-test splits. First, the first 30 samples from each class are used for training, and the last 20 are used for testing. Then, the split is reversed.

The purpose of this task is to evaluate how well the classifier is trained and how well it generalizes to unseen data. The performance is evaluated using a confusion matrix and the error rate.

Part 2 focuses on how the different features affect the linear separability. In this part, the first 30 samples from each class are used for training, and the last 20 are used for testing.

Furthermore, the feature that shows the most overlap between the classes is removed. This process is repeated until only one feature remains.

Here, the performance is also evaluated using a confusion matrix and the error rate.

3.2 Part 1: Design, training, and generalization

Implementation:

The classifier was implemented in Python, and used on the Iris dataset. The data was divided into training and test sets according to the task description. In the first split, the first 30 samples from each class were used for training, while the remaining 20 samples were used for testing. In the second split, this was reversed.

The classifier was trained using the minimum square error (MSE) method. The class labels were encoded using one-hot encoding, and a bias term was added to the feature vectors. The weight matrix was initialized randomly and updated iteratively using gradient descent with a fixed step factor α .

The step factor α was tuned experimentally in order to obtain stable convergence during training. A value of $\alpha = 0.9$ was found to give stable behavior and good classification performance.

Smaller values of α resulted in slower convergence, while larger values caused unstable training.

Results:

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	30	0	0
<i>Versicolor</i>	0	28	2
<i>Virginica</i>	0	1	29

Table 2: Confusion matrix for the training set, split 1
Split 1, training error rate = $0.033 = 3.3\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	20	0	0
<i>Versicolor</i>	0	18	2
<i>Virginica</i>	0	0	20

Table 3: Confusion matrix for the test set, split 1
Split 1, test error rate = $0.033 = 3.3\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	30	0	0
<i>Versicolor</i>	0	25	5
<i>Virginica</i>	0	1	29

Table 4: Confusion matrix for the training set, split 2
Split 2, training error rate = $0.067 = 6.7\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	20	0	0
<i>Versicolor</i>	0	19	1
<i>Virginica</i>	0	0	20

Table 5: Confusion matrix for the test set, split 2
Split 2, test error rate = $0.017 = 1.7\%$

Discussion:

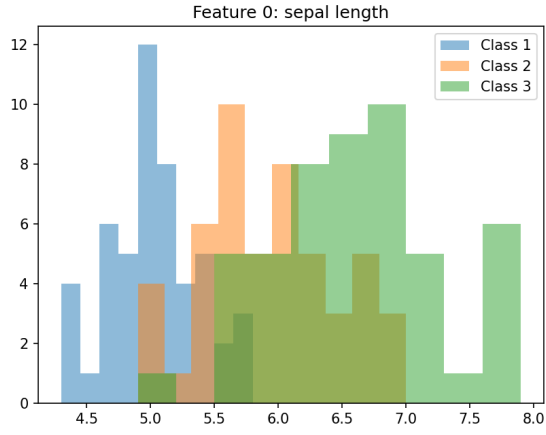
The two train-test splits give different results. While split 1 shows similar error rates for both training and testing, split 2 gives a higher training error rate but a lower test error rate. This suggests that the performance depends on the selected training samples, and that the data set isn't completely uniform.

From the confusion matrices, it can be seen that *Setosa* is always classified correctly. All classification errors occur between *Versicolor* and *Virginica*.

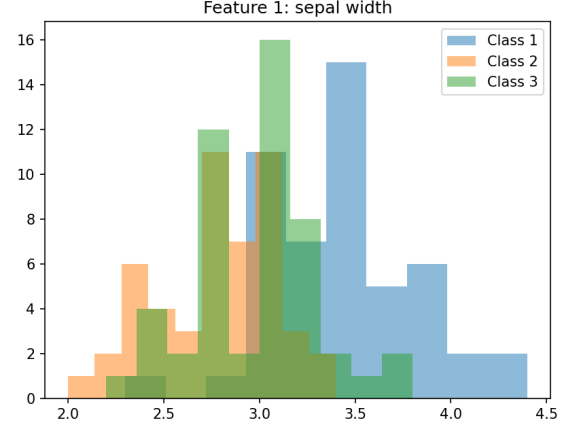
This suggests that *Setosa* is clearly separated from the other two classes, while *Versicolor* and *Virginica* have overlapping feature distributions. Overall, the results show good generalization.

3.3 Part 2. Features and linear separability

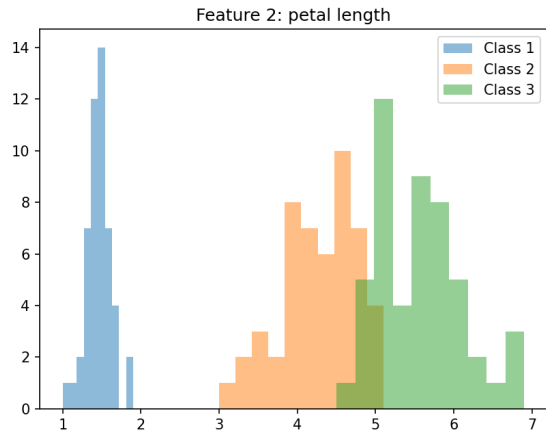
Implementation:



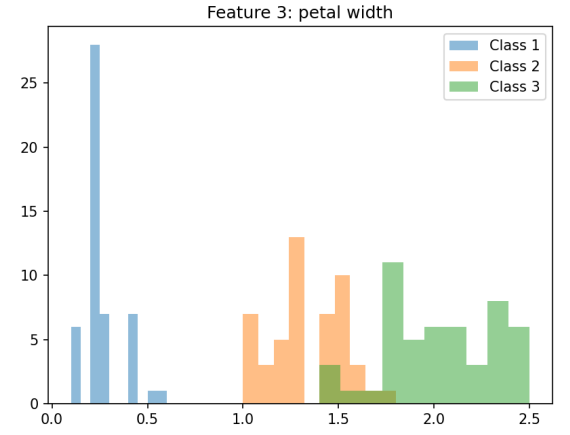
(a) Sepal length and the overlap between the three classes



(b) Sepal width and the overlap between the three classes



(c) Petal length and the overlap between the three classes



(d) Petal width and the overlap between the three classes

Figure 1: Histograms of the four Iris features illustrating the overlap between the three classes

The histograms above were generated for each feature in order to analyze the overlap between the different groups. From the histograms, it can be seen that sepal width shows the greatest overlap between the classes, followed by sepal length, petal length, and finally petal width.

This order was therefore used when removing features in this part. First, the feature with the most overlap was removed, and the classifier was trained using the remaining features. This process was repeated until only one feature remained.

For each case, the classifier was evaluated using the confusion matrix and the error rate.

Results:

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	30	0	0
<i>Versicolor</i>	0	28	2
<i>Virginica</i>	0	1	29

Table 6: Confusion matrix for the training set with 3 features

Training error rate with 3 features = $0.033 = 3.3\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	20	0	0
<i>Versicolor</i>	0	18	2
<i>Virginica</i>	0	1	19

Table 7: Confusion matrix for the test set with 3 features
Test error rate with 3 features = $0.05 = 5\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	30	0	0
<i>Versicolor</i>	0	26	4
<i>Virginica</i>	0	2	28

Table 8: Confusion matrix for the training set with 2 features
Training error rate with 2 features = $0.067 = 6.7\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	20	0	0
<i>Versicolor</i>	0	18	2
<i>Virginica</i>	0	2	18

Table 9: Confusion matrix for the test set with 2 features
Test error rate with 2 features = $0.067 = 6.7\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	30	0	0
<i>Versicolor</i>	0	19	11
<i>Virginica</i>	0	0	30

Table 10: Confusion matrix for the training set with 1 feature
Training error rate with 1 feature = $0.122 = 12.2\%$

True class/Predicted class	<i>Setosa</i>	<i>Versicolor</i>	<i>Virginica</i>
<i>Setosa</i>	20	0	0
<i>Versicolor</i>	0	16	4
<i>Virginica</i>	0	1	19

Table 11: Confusion matrix for the test set with 1 feature
Test error rate with 1 feature = $0.083 = 8.3\%$

Discussion:

The results show that the classification performance decreases as the number of features is reduced. The test error rate increases from 5% with three features to 6.7% with two features and 8.3% with one feature.

It can be seen that *Setosa* is classified correctly in all cases, even when only one feature is used. However, the confusion between *Versicolor* and *Virginica* increases significantly as features are removed.

This indicates that several features are important for separating these two classes. Overall, the results confirm that using more features improves the linear separability of the dataset and leads to better classification performance.

It can also be observed that the difference in performance between using all features (Part 1) and using two or three features is relatively small. This indicates that some features contain overlapping information, and that a reduced feature set is sufficient to achieve good classification performance.

3.4 Conclusion

In this project, a linear classifier based on MSE was implemented and applied to the Iris dataset. The classifier was trained using gradient descent, where the weight matrix was updated iteratively using a step factor α . A sigmoid transformation was applied to the linear output in order to obtain values that can be interpreted as class probabilities.

The results show that the classifier performs well on the dataset. In particular, the Setosa class was perfectly classified in all experiments, while most classification errors occurred between Versicolor and Virginica. This confirms that Setosa is linearly separable from the other classes, whereas the remaining classes have overlapping features. However, the classifier showed good generalization overall, as the test error rates were comparable to the training error rates.

In the second part of the task, the effect of feature selection was investigated. The results showed that reducing the number of features leads to decreased classification performance. However, the difference between using all features and using two or three features was relatively small, indicating that some features contain overlapping information. This suggests that a reduced feature set can still provide good classification results.

Overall, the project provided a good understanding of how linear classifiers are implemented and trained in practice. It also illustrated the importance of feature selection and parameter tuning.

4 Handwritten numbers

This section is composed of three different solutions for classifying handwritten numbers from the MNIST database. The dataset contains 60000 training examples written by 250 different people and 10000 test examples written by 250 others. All data points are preprocessed 28×28 pixel grayscale pictures.

4.1 Task description

The task evaluates two template based classifiers, and explores the effect of clustering. First, a euclidean distance nearest neighbor classifier is implemented using the full training set as templates. Its performance is assessed using a confusion matrix, runtime and error rate.

In the second stage of the task, clustering using k-means is used to reduce the number of templates. The clustering is used to make a 64-point set of centroids, that represents the full training set. The NN classifier is then re-evaluated using the reduced template, comparing performance and efficiency to the former approach.

The last part of the task is building a k-nearest neighbor classifier, where $k = 7$. It is implemented with the full template set, and the efficiency and accuracy is compared to the other approaches.

4.2 Implementation and results

Nearest Neighbor The classifier was implemented in python. The euclidean distance was implemented with `cdist` from the `scipy` library, which implements the Euclidean distance shown in Equation 1 in an efficient manner.[2] To reduce the computational cost, the test data was processed in batches.

The performance of the model is shown by the confusion matrix in 2.

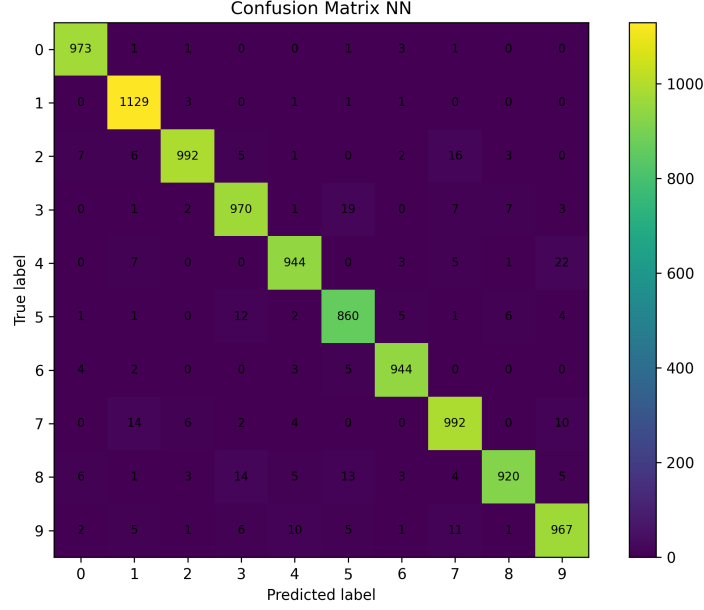


Figure 2: Confusion matrix for the NN classifier on the test set. Rows represent true labels and columns represent predicted labels.

The classifier had an accuracy of 96.91% and a runtime of 171.86s. This corresponds to an error rate of 3.09%.

The following numbers were misclassified and correctly classified:

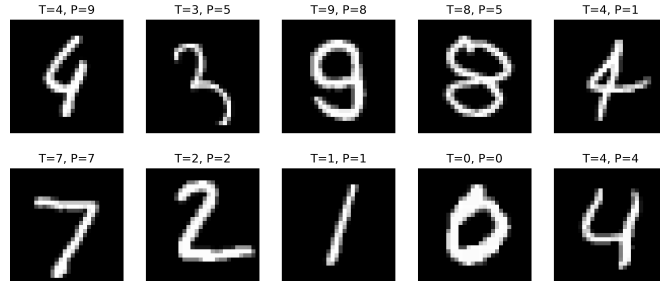


Figure 3: Top 5 classified wrong and bottom 5 correctly. T shows true label and P is classification.

We can clearly see that the numbers that are classified wrong bear strong resemblance with another number. They are either written sloppily or have an irregular orientation. Some of the misclassified digits are ambiguous even for a human, but other are clearly just written in a slightly different orientation or irregular shape. From the confusion matrix it becomes clear that the classifier struggles to separate the same numbers that we as humans also mix. Two examples are 7 and 2, and 7 and 1.

Nearest Neighbor with clustering

The NN classifier with the clustering is based on the same classifier but uses the centroid template instead of the whole training set. Firstly the $M = 64$ clusters are trained using sklearn's k-means algorithm. The algorithm uses the principles shown in the theory. Thereafter the same NN algorithm is ran with the test-data and centroids made by k-means. [3]

The performance of the algorithm is shown in the following confusion matrix:

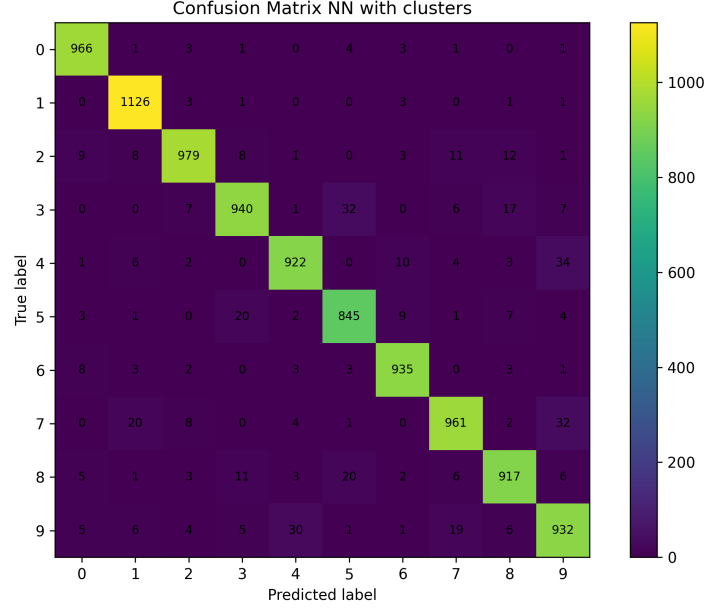


Figure 4: Confusion matrix for the NN classifier with clustering. Rows represent true labels and columns represent predicted labels.

The accuracy was calculated to 95.23% and the runtime was measured to be 20.41s. This corresponds to an error rate of 4.77%.

K-NN classifier The KNN classifier is implemented by computing distances in the same way as with the NN classifier. For each sample, the $k = 7$ nearest neighbors are found, and the predicted label is determined by majority voting among these neighbors.

The performance of the KNN classifier is shown in the following confusion matrix:

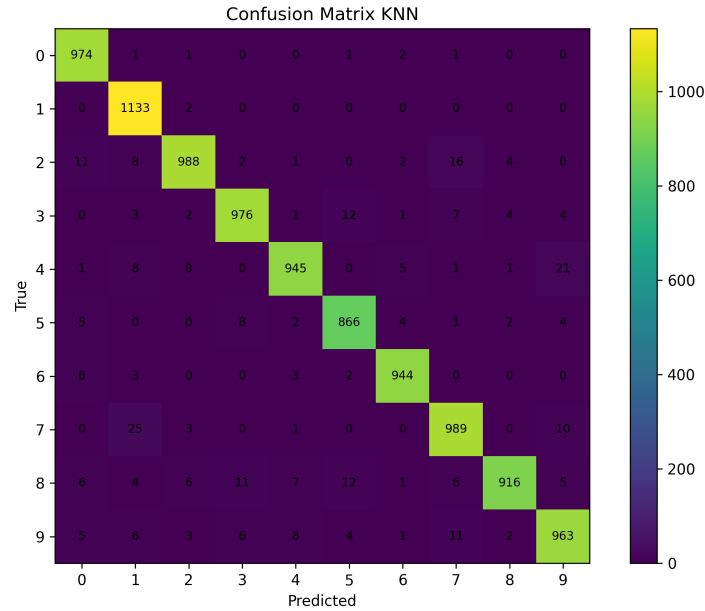


Figure 5: Confusion matrix for the KNN classifier. Rows represent true labels and columns represent predicted labels.

The accuracy was calculated to 96.94% and the runtime was measured to 206.66s. This

corresponds to an error rate of 3.06%.

4.3 Discussion

The results show clear trade-offs between accuracy and computational efficiency for the three algorithms.

The standard nearest neighbor classifier achieves a high accuracy of 96.91%, by computing against the full training set. However, this comes with a significant computational cost, resulting in a runtime of 171.86s. The confusion matrix indicates that most of the errors occur between visually similar digits. This is consistent with the limitations of distance-based classifiers, and with the conclusions drawn from the examples shown of wrong classifications.

Classification with clustering, with k-means, reduces the number of data-points significantly, resulting in a substantially lower runtime. The runtime improves by 88%, which means it is reduced to 20.41s. However, the accuracy drops to 95.23%, showing that some accuracy is lost when each class is represented by 64 data points. The confusion matrix also shows an increase in misclassifications between similar digits.

The KNN classifier with $K = 7$ achieves the highest accuracy of 96.94%, slightly outperforming the NN classifier. This improvement is due to the majority voting, which gives a reduced sensitivity to outliers and noise. However, it requires more calculation, and as a result has a runtime of 206.66s. The performance gain is marginal, resulting in diminishing return relative to the increased cost.

The confusion matrices reveals consistent patterns across all three classifiers. Several off-diagonal elements show systematic confusion between specific digits.

In particular, digits such as 7 and 2, 9 and 4, 8 and 5, 5 and 3, as well as 7 and 1, are frequently misclassified. This can be explained by similarities in their shapes, especially when written with varying styles or orientations. For example, a handwritten 7 without a crossbar can resemble a 1, while a 9 with a straight tail resembles a 4.

Comparing the three methods, the NN and KNN classifiers show similar confusion patterns, but KNN slightly reduces the number of misclassifications due to the majority voting. In contrast, the clustering-based approach introduces more confusion between similar digits, as the reduced number of centroids cannot fully capture the variability within each class. Overall, the confusion matrices confirm that the main source of error is not random, but arises from inherent similarities between certain digit classes.

4.4 Conclusion

This project has demonstrated how template based classification methods perform on the MNIST dataset. The methods use distance measures in feature space to quantify similarity between samples. The results confirm that simple distance-based classifiers can achieve high accuracy.

The project highlighted the importance of efficient computation. The use of vectorized computation such as `cdist` and processing in batches, was necessary to make the nearest neighbor classifier efficient enough for practical application.

The results show both positive and negative aspects of the methods used. The NN classifier provides strong accuracy by using the full training set, but suffers from high computational cost. The clustering-based approach significantly reduces runtime. However, this comes at the cost of reduced accuracy, as some variance within each class are lost. The KNN classifier achieves the highest accuracy, demonstrating increased robustness. On the other hand, it has an even higher computational cost than NN, leading to diminishing returns in performance/cost metrics.

The confusion matrices shows that the main source of classification errors is inherent similarities between certain digits. This indicates a limitation of distance based methods, as they rely purely on pixel by pixel similarity, and don't capture high level structural features.

Through this project, a deeper understanding of template based classifiers was developed, including their strengths, limitations and costs. In particular, the importance of efficient implementation, the effect of clustering, and the gains of KNN implementation over NN were key learning outcomes.

References

- [1] M. H. Johnsen, “Compendium - part iii - classification,” 2017. NTNU course material.
- [2] SciPy Developers, “`scipy.spatial.distance.cdist`.” <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html>, 2026. Accessed: 2026-04-23.
- [3] scikit-learn developers, “`sklearn.cluster.kmeans`.” <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>, 2026. Accessed: 2026-04-23.